

WEB DEVELOPMENT – III

HTTP Fundamentals

Session 2 — Study Notes

Topic: HTTP — Requests, Responses, Methods, Status Codes, Headers

Instructor: Love Porwal

K.R. Mangalam University — Sohna Road, Gurugram

1. What Is HTTP?

Order food on Zomato or Swiggy, and press "Order Now." In that instant, your phone has to tell a distant server exactly what you want, and that server has to reply confirming the order, all in a language both sides already agree on. That shared language is HTTP.

HTTP — HyperText Transfer Protocol — is the foundation of communication on the web. It is a set of rules that defines how messages are formatted and transmitted between a client and a server, so that no matter what browser, phone, or programming language either side uses, they can still understand each other. Every time a website is opened, an app refreshes a feed, or a form is submitted, HTTP is quietly doing the work underneath.

This matters well beyond browsing. When a form is submitted on a website, HTTP is what carries that data to the backend. When Netflix streams a movie, HTTP — specifically its secure version, HTTPS — carries the video frames across the network. Every request an app makes to check for new messages, every response that updates a dashboard, is an HTTP exchange. Understanding HTTP is what makes it possible to build APIs, debug network issues, and reason about why an app feels slow or broken.

2. The HTTP Request & Response Cycle

Typing a URL and seeing a page appear feels instantaneous, but it hides a precise sequence of steps. Each step has to succeed before the next one can begin:

1. User action — you type a URL or click a link in the browser.
2. DNS resolve — the domain name (like `api.college.com`) is looked up and converted into an IP address, since machines route traffic by IP, not by name.
3. TCP connect — the browser opens a connection to the server, usually on port 80 (HTTP) or port 443 (HTTPS).
4. HTTP request — the browser sends a GET, POST, or other request, carrying headers and, if needed, a body.
5. Server process — the server runs its code, queries the database if required, and prepares a response.

What Happens Between a Click and a Loaded Page

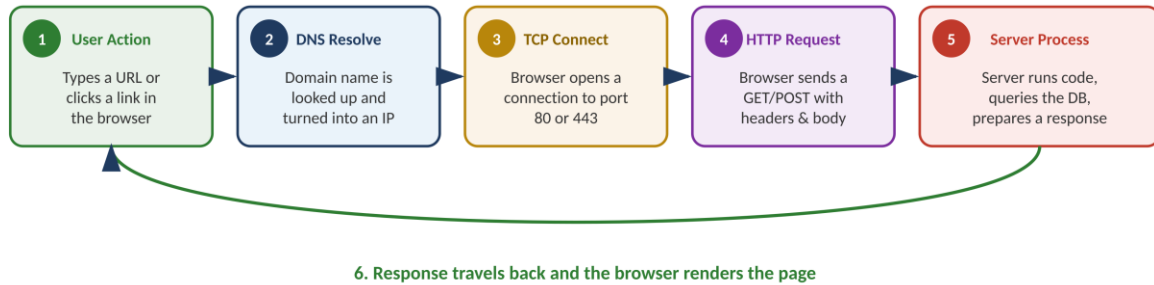


Figure 2.1 — The five steps between a click and a server actually doing something, closed by the response coming back.

Once the server has prepared its answer, the response travels back over the same connection, and the browser renders it — displaying a webpage, updating an app's screen, or handing data to whatever code requested it. Nothing about this cycle changes whether you are loading a college portal or placing a food order; only the specific request and the data in the response differ.

What an HTTP Request Actually Contains

An HTTP request is simply a structured message sent from a client to a server, asking for information or requesting an action. Every request, regardless of what it's for, is built from four parts:

- **Method** — tells the server what you want to do (GET, POST, PUT, PATCH, DELETE).
- **URL** — tells the server where the request should go, such as `https://api.college.com/students`.
- **Headers** — carry extra information about the request, such as `Content-Type: application/json` or `Authorization: Bearer xyz123`.
- **Body (optional)** — contains the actual data being sent, such as `{ "name": "Rahul", "age": 21 }`.

Anatomy of an HTTP Request

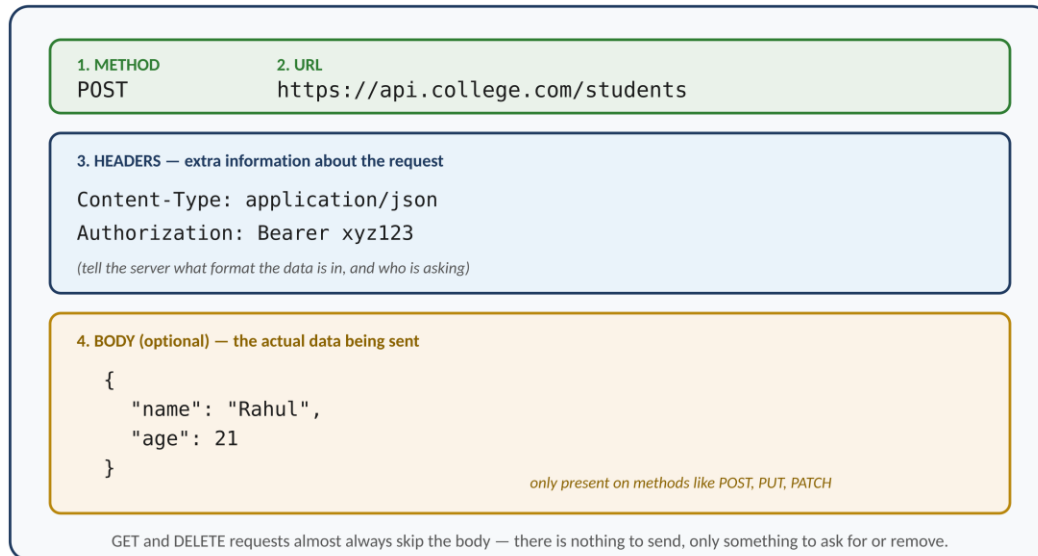


Figure 2.2 — The same four parts, laid out as a real request would look.

Not every request needs all four parts. A GET request, which only asks for data, typically has no body at all — there's nothing to send, only something to retrieve. A POST request that creates a new student record, on the other hand, needs a body to carry the student's details.

3. HTTP Methods

HTTP methods are verbs — they tell the server what action you intend to perform on a resource, independent of what that resource actually is. A well-designed API uses these verbs consistently, so that GET always means "give me data" and never "change something," no matter which endpoint it's called on.

Method	Purpose	Has Body?	Idempotent?	Example Use
GET	Retrieve data	No	Yes	Fetch user list
POST	Create a new resource	Yes	No	Register a user
PUT	Replace entire resource	Yes	Yes	Update full profile
PATCH	Partially update a resource	Yes	Yes	Change only email
DELETE	Remove a resource	No	Yes	Delete a post
HEAD	Like GET but no response body	No	Yes	Check if a file exists
OPTIONS	Describe available methods (CORS)	No	Yes	CORS preflight check

Idempotency, in Plain Terms

A method is idempotent if calling it once has exactly the same effect as calling it five times in a row. GET is idempotent because asking for the same data repeatedly never changes anything. DELETE is idempotent because deleting an already-deleted post still leaves it deleted — the end state doesn't change no matter how many times you repeat it. POST is the odd one out: calling it twice usually creates two resources instead of one, which is exactly why refreshing a page after submitting a form sometimes triggers a warning about resubmitting data.

PUT vs PATCH — A Common Point of Confusion

Both PUT and PATCH update an existing resource, but they disagree about how much of it you must send. PUT expects the complete replacement — if a user record has five fields and a PUT request only includes one, the other four are typically wiped out or reset. PATCH, by contrast, updates only the fields actually included in the request, leaving everything else untouched. For small, partial changes — updating just an email address, say — PATCH is the safer and more common choice.

4. HTTP Status Codes

A status code is a three-digit number the server attaches to every response, telling the client at a glance whether the request succeeded, failed, or needs further action. Reading status codes correctly is central to error handling — a frontend that ignores them will show a blank screen instead of a meaningful message when something goes wrong.

The first digit of a status code tells you which family it belongs to, and that alone is often enough to know how to react to it:

HTTP Status Code Families		
1xx	Informational Request received, server is continuing to process it	100 Continue
2xx	Success The request was received, understood, and accepted	200 OK 201 Created · 204 No Content
3xx	Redirection Further action is needed to complete the request	301 Moved Permanently 302 Found · 304 Not Modified
4xx	Client Error Something about the request itself is wrong	400 Bad Request · 401 Unauthorized 403 Forbidden · 404 Not Found 409 Conflict
5xx	Server Error The request was fine, but the server failed to handle it	500 Internal Server Error 502 Bad Gateway · 503 Unavailable

Figure 4.1 — The five status code families, from purely informational to a server-side failure.

A few of these are worth knowing by number, since they appear constantly in real applications. 200 OK is the default success response. 201 Created confirms that a POST request successfully made something new. 400 Bad Request means the request itself was malformed — missing fields, invalid JSON. 401 Unauthorized means the server needs proof of identity that wasn't provided. 403 Forbidden means the identity is known but not permitted to do this. 404 Not Found means the specific resource — a wrong URL, a deleted item — simply isn't there. 500 Internal Server Error means the fault lies with the server's own code, not with anything the client did.

5. HTTP Headers

Headers are key-value pairs attached to a request or response, carrying metadata — information about the data — rather than the data itself. Picture sending a courier parcel: the parcel holds the actual gift, but a label is stuck on the outside listing the sender's name, the receiver's address, a "fragile" warning, and the weight. None of that label changes what's inside the box, yet it completely determines how the courier handles it — which truck it goes on, how carefully it's carried, where it ends up. HTTP headers play exactly that role: they never alter the actual data, but they tell the server or the browser how to treat it.

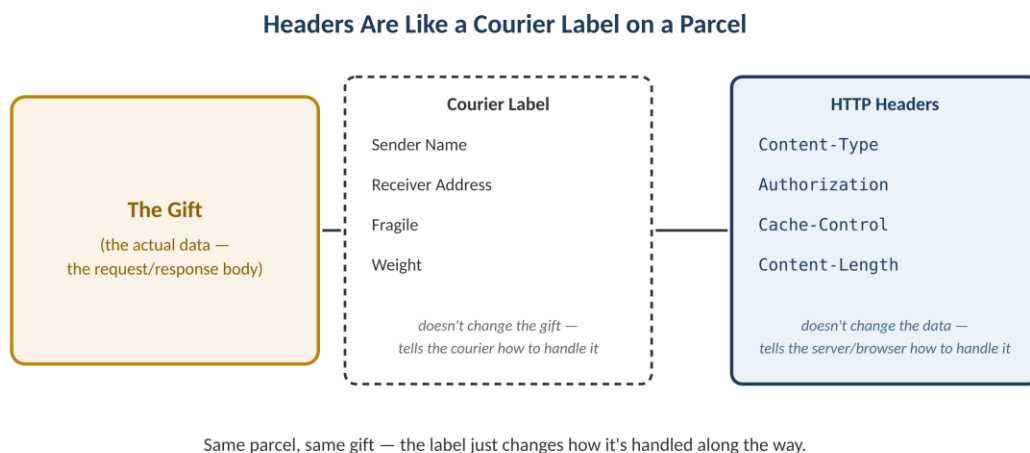


Figure 5.1 — The same idea, side by side: a courier label and an HTTP header block.

Some headers show up constantly enough to be worth memorizing:

Header	What It Does
Content-Type	Tells the receiver what format the body is in — application/json, text/html, multipart/form-data.
Authorization	Carries credentials, most often as Bearer <token>, proving who is making the request.
Accept	Tells the server what response formats the client is willing to receive.

Header	What It Does
Cache-Control	Controls whether and how long a response may be cached before it must be re-fetched.
User-Agent	Identifies the browser, app, or device making the request.
Content-Length	States the size of the body in bytes, so the receiver knows when the data ends.

Content-Type deserves special attention because it is so easy to get wrong and so disruptive when it is. When a request's body is JSON but the Content-Type header still says something else, the server often fails to parse the body at all — not because the data is wrong, but because the server was never told how to read it. The header and the actual body format must always match.

6. Key Takeaways

To create a new resource, the correct method is POST — sent to an endpoint such as `/api/users`, with the new user's data in the body. GET only retrieves; PUT replaces something that already exists; DELETE removes it. Using the semantically correct method matters even when a wrong one would technically still return some result.

A 404 status code means specifically that the resource could not be found — a wrong URL or a deleted item — and nothing more. It is easy to confuse with 400 (a badly formatted request), 401 (missing authentication), or 500 (a bug on the server itself); each of these describes a different failure and calls for a different fix.

Setting Content-Type: `application/json` on a request is not a formality — it directly tells the server how to parse the body being sent. Without it, a perfectly valid JSON body can still fail to parse correctly, because the server was never told what format to expect.

PUT and PATCH are not interchangeable. PUT expects the full resource and can silently erase fields that are left out; PATCH updates only the fields explicitly included, which makes it the safer default for partial edits.

7. Summary

- HTTP is the shared set of rules that lets clients and servers exchange messages in a format both understand.
- Every page load follows the same cycle: user action, DNS resolve, TCP connect, HTTP request, server processing, and a response back.
- A request is built from four parts — method, URL, headers, and an optional body.
- HTTP methods are verbs: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes.

- Status codes summarize the outcome of a request at a glance: 2xx success, 3xx redirection, 4xx client error, 5xx server error.
- Headers carry metadata — like a courier label — that shapes how data is handled without changing the data itself.

8. References

- Node.js Official Documentation — runtime, modules, APIs, file system, HTTP, and Express integration basics.
- W3Schools — Node.js Tutorial: beginner-friendly explanations with hands-on examples.
- freeCodeCamp — Node.js Course: free, project-based learning for Node.js, Express.js, and REST APIs.
- HTTP Status — a reference for every HTTP status code (httpstatuses.com).